ACADEMIC CENTRAL

PRACTICE EXAMINATION

NC(V) LEVEL 3

INTRODUCTION TO TECHNICAL PROGRAMMING

PRACTICE EXAM 3 · MARKING GUIDELINE

MARKS: 100

Marking guideline for the Academic Central NC(V) Level 3 Introduction to Technical Programming practice exam 3.

Written by Academic Central for practice, following the NC(V) subject guidelines for NC(V) Level 3. It is not a Department of Higher Education and Training marking guideline.

How to mark: each ✓ is one mark. Accept any correct, equivalent code or pseudocode. Do not penalise small syntax slips (e.g. a missing semicolon) more than once per question unless syntax is being tested. Give follow-through marks in trace tables.

SECTION A

QUESTION 1

1.1 1.1.1 B ✓ 1.1.2 C ✓ 1.1.3 B ✓ 1.1.4 C ✓ 1.1.5 A ✓ 1.1.6 B ✓ 1.1.7 D ✓ 1.1.8 C ✓ 1.1.9 D ✓ 1.1.10 A ✓ (10)

1.2 1.2.1 True ✓ 1.2.2 False ✓ 1.2.3 False ✓ 1.2.4 True ✓ 1.2.5 False ✓ 1.2.6 True ✓ 1.2.7 True ✓ 1.2.8 False ✓ 1.2.9 False ✓ 1.2.10 True ✓ (10)

1.3 1.3.1 C ✓ 1.3.2 I ✓ 1.3.3 E ✓ 1.3.4 G ✓ 1.3.5 B ✓ 1.3.6 J ✓ 1.3.7 D ✓ 1.3.8 F ✓ 1.3.9 A ✓ 1.3.10 H ✓ (10)

TOTAL SECTION A: [30]

SECTION B

QUESTION 2

2.1 Sequence ✓, selection (decision) ✓, repetition (iteration / loops) ✓ (3)

2.2 Set total to 0 ✓; repeat 10 times ✓: input reading ✓, add reading to total ✓; average = total ÷ 10 ✓; if average > 60 then display "OVERHEAT" ✓, e.g. (6)

```
total ← 0
FOR count ← 1 TO 10
    INPUT reading
    total ← total + reading
END FOR
average ← total / 10
IF average > 60 THEN
    DISPLAY "OVERHEAT"
END IF
```

2.3 Rows (i, b, a): 1, 1, 2 ✓; 2, 3, 4 ✓; 3, 7, 8 ✓; 4, 15, 16 ✓. Output: 16 15 ✓ (5)

[14]

QUESTION 3

3.1 (a) int ✓ (b) float / double ✓ (c) bool ✓ (d) string ✓ (4)

3.2 (a) 3 ✓ (b) 2 ✓ (c) 3.4 ✓ (d) true / 1 ✓ (e) 7 ✓ (5)

3.3 = assigns a value to a variable, e.g. speed = 100; ✓. == tests whether two values are equal, e.g. if (speed == 100) ✓. (2)

3.4 const declaration ✓; declare and read speed ✓✓; if test ✓; assignment ✓, e.g. (5)

```
const int MAX_SPEED = 255;
int speed;
cin >> speed;
if (speed > MAX_SPEED) {
  speed = MAX_SPEED;
}
```

[16]

QUESTION 4

4.1 switch (mode) ✓; case 1 ✓, case 2 and case 3 ✓ with the correct output; break after each case ✓; default for "Invalid" ✓; correct braces / syntax ✓, e.g. (6)

```
switch (mode) {
  case 1: cout << "Forward"; break;
  case 2: cout << "Reverse"; break;
  case 3: cout << "Stop"; break;
  default: cout << "Invalid";
}
```

4.2 A while loop tests the condition first, so its body may run zero times ✓✓; a do-while loop tests the condition after the body, so it always runs at least once ✓. Example: a menu that must be shown at least once / asking for input until it is valid ✓. (4)

4.3 * (line 1) ✓ ** (line 2) ✓ *** (line 3) ✓ (3)

4.4 Declare variable ✓; loop that repeats ✓; prompt and input inside the loop ✓; correct condition for values outside 0–400 ✓; display the valid value after the loop ✓, e.g. (5)

```
int distance;
do {
  cout << "Enter distance (0-400): ";
  cin >> distance;
} while (distance < 0 || distance > 400);
cout << "Distance: " << distance;
```

4.5 A loop whose condition never becomes false, so it never stops ✓. Cause, e.g. forgetting to change the loop variable / wrong condition / using = instead of == ✓ (2)

[20]

QUESTION 5

5.1 Function header with float parameter and float return type ✓; correct calculation ✓; return statement ✓; call in main ✓; display the result ✓, e.g. (5)

```
float cmToInch(float cm) {
  return cm / 2.54;
}

int main() {
  cout << cmToInch(30) << " inches";
```

```
  return 0;
}
```

5.2 Start with the first element as the largest ✓; loop through the array ✓; compare each element ✓; update the largest ✓; display after the loop ✓, e.g. (5)

```
float largest = readings[0];
for (int i = 1; i < 5; i++) {
  if (readings[i] > largest) {
    largest = readings[i];
  }
}
cout << "Largest: " << largest;
```

5.3 (a) 7 ✓ (b) name[0] (or name.at(0)) ✓ (2)

5.4.1 voltage ✓ (1)

5.4.2 A constructor ✓; it runs automatically when an object is created and gives voltage its starting value ✓ (2)

5.4.3 Create the object with 6.5 ✓; if (object.isLow()) ✓; display "Charge now" ✓, e.g. (3)

```
Battery pack(6.5);
if (pack.isLow()) {
  cout << "Charge now";
}
```

5.4.4 Encapsulation / data hiding ✓: code outside the class cannot change voltage directly, so it can only be set or used through the class's own functions, which protects the data from mistakes ✓ (2)

[20]

TOTAL SECTION B: [70]

TOTAL: 100

END OF MARKING GUIDELINE